Guia Definitivo de Consulta: Windows Forms via Código (LPR2)

Baseado nas Aulas 4, 5, 6 e Projetos Práticos (`CadastroAlunos`, `CatalogoProdutos` e `FichaInscricao`)

---

1. Comandos de Terminal (CLI) e Estrutura do Projeto

Comandos no PowerShell do VS Code

```shell
dotnet --info                                  # Verifica informações do .NET instalado
dotnet --list-sdks                             # Lista os SDKs instalados (deve ter o 8.0.x)
dotnet new list winforms                       # Verifica se o template winforms existe
dotnet new winforms -n NomeProjeto -f net8.0 # Cria o projeto Windows Forms no .NET 8.0
cd NomeProjeto                                 # Entra na pasta do projeto
code .                                         # Abre a pasta atual no VS Code
dotnet build                                   # Compila o projeto e verifica erros
dotnet run                                     # Compila (se necessário) e executa a aplicação
```

Limpeza Inicial e Estrutura de Arquivos

Ao criar o projeto pelo terminal, **apague** os arquivos gerados pelo template (`Form1.cs`, `Form1.Designer.cs` e `Form1.resx`) e organize em **4 arquivos principais**:

1. **`NomeProjeto.csproj`**: Arquivo XML de configuração (`OutputType` = `WinExe`, `TargetFramework` = `net8.0-windows`, `UseWindowsForms` = `true`).
2. **`Program.cs`**: Ponto de entrada que inicia a janela principal.
3. **`Entidade.cs`** (ex: `Aluno.cs`, `Produto.cs`): Classe de modelo de dados.
4. **`CadastroForm.Layout.cs`**: Parte 1 da classe parcial (`partial class`). Declara os campos dos controles (`private readonly`) e o método `MontarTela()`.

5. **`CadastroForm.cs`**: Parte 2 da classe parcial (`partial class : Form`). Contém a `BindingList`, o estado de edição (`alunoEmEdicao`), o construtor, `ConectarEventos()` e a lógica (salvar, validar, selecionar, excluir).

Código Padrão do `Program.cs`

```
C#
namespace NomeDoProjeto;

internal static class Program
{
    [STAThread] // Configura a thread principal para recursos gráficos do Windows
    private static void Main()
    {
        ApplicationConfiguration.Initialize(); // Aplica configuração gráfica
inicial
        Application.Run(new CadastroForm());   // Abre o formulário e inicia o
laço de mensagens
    }
}
```

---

2. Propriedades Universais (Comuns à maioria dos Controles)

Estes atributos podem ser definidos na criação (`new() { ... }`) ou depois via código (`controle.Propriedade = valor;`):

Propriedade / Método	Tipo / Valores Usados	O que faz
`Text`	`string`	Define ou lê o texto exibido no controle (título da janela, texto do botão, conteúdo do `TextBox` ou `Label`).
`Dock`	`DockStyle.Fill DockStyle.Bottom`	Preenche todo o espaço disponível da célula/janela (`Fill`) ou cola na base da janela (`Bottom`).
`Anchor`	`AnchorStyles.Left AnchorStyles.Right`	Mantém o controle ancorado (preso) à esquerda ou direita dentro da célula do layout.

Propriedade / Método	Tipo / Valores Usados	O que faz
`AutoSize`	`bool` (`true` / `false`)	Ajusta automaticamente o tamanho do controle ao seu conteúdo (muito usado em `Label`, `Button`, `CheckBox`, `RadioButton`, `FlowLayoutPanel`).
`TabIndex`	`int` (`0`, `1`, `2`...)	Define a ordem de navegação quando o usuário aperta a tecla `Tab` (começa em `0` dentro de cada contêiner).
`Margin`	`new Padding( todos ) new Padding(esq, topo, dir, base)`	Espaço **externo** ao redor do controle. Ex: `new Padding(3, 3, 24, 3)` deixa `24px` na direita para caber o ícone do `ErrorProvider`.
`Padding`	`new Padding( todos ) new Padding(esq, topo, dir, base)`	Espaço **interno** dentro do controle/contêiner (ex: `new Padding(12)` no `TableLayoutPanel`).
`Enabled`	`bool` (`true` / `false`)	Habilita ou desabilita o controle para interação (ex: `btnExcluir.Enabled = false;`).
`Width`	`int` (ex: `200`)	Define uma largura fixa para o controle (usado quando não está com `Dock = DockStyle.Fill`).
`MinimumSize`	`new Size(largura, altura)`	Define o tamanho mínimo permitido para o controle ou janela.
`Font`	`new Font("Segoe UI", 12F, FontStyle.Bold)`	Altera a família da fonte, tamanho em pontos (`F`) e estilo (`FontStyle.Bold`).

Propriedade / Método	Tipo / Valores Usados	O que faz
`ForeColor`	`Color.DarkBlue Color.DarkGreen`	Altera a cor do texto do controle.
`BackColor` / **`BackgroundColor`**	`Color.White`, `Color.LightCoral`, `Color.LightYellow`, `SystemColors.Window`	Altera a cor de fundo.
`Focus()`	Método: `controle.Focus();`	Leva o cursor/foco diretamente para aquele controle.
`BringToFront()`	Método: `controle.BringToFront();`	Traz o controle para a frente na ordem de empilhamento visual (essencial após adicionar barras e o `TableLayoutPanel` no `Form`).

---

3. Catálogo Completo de Componentes e Elementos

3.1. `Form` (A Janela Principal)

A própria classe `CadastroForm` herda de `Form`. Suas propriedades são configuradas diretamente dentro do método `MontarTela()`:

- **Atributos configurados:**
  - `Text = "Título da Janela";`
  - `ClientSize = new Size(760, 480);` (Tamanho da área útil interna da janela: largura, altura).
  - `MinimumSize = new Size(600, 400);` (Impede que o usuário encolha demais a janela).
  - `StartPosition = FormStartPosition.CenterScreen;` (Abre a janela centralizada na tela).
  - `AutoScaleMode = AutoScaleMode.Font;` (Escala a interface proporcionalmente à fonte do sistema).
  - `AcceptButton = btnSalvar;` (Faz a tecla `Enter` acionar o botão Salvar automaticamente).
  - `MainMenuStrip = menuPrincipal;` (Define qual é o `MenuStrip` principal da janela).
  - `Controls.Add(controle);` (Adiciona um controle/painel diretamente à janela).

- **Métodos:**
  - `Close();` (Fecha a janela e encerra a aplicação se for o formulário principal).

3.2. Controles de Texto (`Label`, `TextBox` e Similares)

`Label` (Texto estático / Rótulo / Status)

- **Uso:** Identificar campos e exibir mensagens ou resultados na tela.
- **Atributos usados:**
  - `Text`, `AutoSize = true`, `Dock`, `Anchor`, `Font`, `ForeColor`, `Padding`, `Margin`.
  - `TextAlign = ContentAlignment.TopRight;` (Alinha o texto dentro do espaço do `Label`).
- **Evento de Acessibilidade (Dica de Prova):**
  - Clicar no rótulo foca no campo correspondente:

```
C#
lblNome.Click += (s, e) => txtNome.Focus();
```

`TextBox` (Entrada de texto curto)

- **Uso:** Receber textos digitados pelo usuário (nome, código, preço, pesquisa).
- **Atributos na Criação/Configuração:**
  - `Dock = DockStyle.Fill`
  - `MaxLength = 100` (Limita a quantidade máxima de caracteres digitados).
  - `TabIndex = 0`
  - `PlaceholderText = "Digite seu nome completo"` (Texto cinza de dica que some ao digitar).
  - `Margin = new Padding(3, 3, 24, 3)`
  - *Outros citados nos slides:* `ReadOnly` (`bool`), `Multiline` (`bool`), `PasswordChar` (`char`).
- **Leitura e Métodos:**
  - **Ler texto limpo:** `string nome = txtNome.Text.Trim();` (`Trim()` remove espaços em branco antes e depois).
  - **Limpar campo:** `txtNome.Clear();`
  - **Focar:** `txtNome.Focus();`
- **Eventos usados:**
  - `txtPesquisa.TextChanged += PesquisarProduto;` (Dispara **toda vez** que o texto muda, ideal para filtro em tempo real ou cálculo automático).

Outros controles de texto mencionados na Aula 6 (Conceito Teórico):

- **`MaskedTextBox`**: Entrada com formato/máscara fixa (ex: CEP ou telefone).
- **`RichTextBox`**: Texto longo ou com formatação (ex: observações).
- **`LinkLabel`**: Texto clicável com aparência de link (para abrir uma página ou recurso).

3.3. Controles de Escolha (`ComboBox`, `CheckBox`, `RadioButton`)

`ComboBox` (Lista Suspensa)

- **Uso:** Escolher uma única opção de uma lista pré-definida (ex: Curso, Categoria).
- **Atributos na Criação/Configuração:**
  - `DropDownStyle = ComboBoxStyle.DropDownList` (**Essencial:** impede que o usuário digite textos novos; obriga a escolher da lista).
  - `Dock = DockStyle.Fill` ou `Anchor = AnchorStyles.Left` com `Width = 200`.
  - `TabIndex`, `Margin`.
- **Como Preencher Opções (em `MontarTela`):**

```
C#
cmbCategoria.Items.Add("Todas"); // Adiciona 1 item
cmbCurso.Items.AddRange(new object[] {
    "Administração", "Agronomia", "Sistemas de Informação"
}); // Adiciona vários itens de uma vez
```

- **Propriedades de Manipulação e Leitura:**
  - `SelectedIndex`: Índice numérico do item escolhido (`0` é o primeiro; `-1` significa **nenhum selecionado**).
    - Para limpar/desmarcar: `cmbCurso.SelectedIndex = -1;` (ou `0` para voltar ao primeiro item).
    - Para validar se escolheu: `if (cmbCurso.SelectedIndex < 0) { /* Erro! */ }`
  - `SelectedItem`: O objeto selecionado.
    - **Para ler como string segura:** `string curso = cmbCurso.SelectedItem?.ToString() ?? "";`
    - **Para carregar na edição:** `cmbCurso.SelectedItem = alunoEmEdicao.Curso;`
- **Eventos usados:**
  - `cmbCategoria.SelectedIndexChanged += PesquisarProduto;` (Dispara quando o usuário troca a opção selecionada).

`CheckBox` (Caixa de Marcação Independente)

- **Uso:** Opções independentes de Sim/Não (`bool`), como "Receber avisos", "Somente em estoque", "Produto em Promoção".
- **Atributos na Criação/Configuração:**
  - `Text = "Receber avisos"`, `AutoSize = true`, `TabIndex`, `Anchor`.
  - `Checked = true` (ou `false` — define o estado marcado/desmarcado).
- **Leitura e Escrita:**
  - **Ler:** `bool avisos = chkAvisos.Checked;`
  - **Alterar:** `chkAvisos.Checked = alunoEmEdicao.ReceberAvisos;`
- **Eventos usados:**
  - `chkPromocao.CheckedChanged += MudancaPromocao;` (Dispara quando marca ou desmarca a caixinha).

`RadioButton` (Escolha Exclusiva em Grupo)

- **Uso:** Escolher **apenas uma** alternativa entre várias do mesmo contêiner (ex: "Presencial" vs "Remota", "Ativo" vs "Inativo").
- **Regra de Ouro:** Para que os `RadioButton` funcionem juntos (marcar um desmarca o outro), eles devem ser adicionados dentro do **mesmo contêiner** (como um `FlowLayoutPanel` ou `GroupBox`).
- **Atributos na Criação/Configuração:**
  - `Text = "Presencial"`, `AutoSize = true`, `Checked = true` (deixe um deles como `true` por padrão), `TabIndex`, `Margin`.
  - `TextAlign = ContentAlignment.MiddleCenter`
- **Leitura e Escrita:**
  - **Ler:** `bool ehPresencial = rdbPresencial.Checked;`
  - **Carregar na edição:**

```
C#
if (produtoEmEdicao.Ativo)
    rdbAtivo.Checked = true;
else
    rdbInativo.Checked = true;
```

Outros controles de escolha mencionados na Aula 6 (Conceito Teórico):

- **`ListBox`**: Exibe várias opções visíveis simultaneamente; aceita seleção única ou múltipla.
- **`CheckedListBox`**: Lista em que cada opção possui um `CheckBox` próprio independente.

3.4. Datas, Números e Escalas (`DateTimePicker`, `NumericUpDown`)

`DateTimePicker` (Seletor de Data)

- **Uso:** Selecionar uma data válida no calendário sem precisar validar texto.
- **Atributos na Criação/Configuração:**
  - `Format = DateTimePickerFormat.Short` (Exibe a data no formato curto padrão: `dd/mm/aaaa`).
  - `Dock = DockStyle.Fill` ou `Anchor = AnchorStyles.Left` com `Width = 200`.
  - `TabIndex`, `Margin`.
- **Leitura e Escrita (`Value`):**
  - **Ler apenas a data (sem horas):** `DateTime dataNasc = dtpDataNasc.Value.Date;`
  - **Resetar no `PrepararNovo`:** `dtpDataNasc.Value = DateTime.Today;` (ou `DateTime.Now`).
  - **Carregar na edição:** `dtpDataNasc.Value = alunoEmEdicao.DataNascimento;`

`NumericUpDown` (Seletor Numérico com Setinhas)

- **Uso:** Receber números inteiros ou decimais dentro de limites definidos (ex: Dependentes, Estoque, % de Desconto).
- **Atributos na Criação/Configuração:**
  - `Minimum = 0` (Valor mínimo aceito).
  - `Maximum = 10` (Valor máximo aceito).
  - `Increment = 1` (De quanto em quanto aumenta ao clicar na setinha).
  - `DecimalPlaces = 0` (Quantidade de casas decimais; `0` para números inteiros).
  - `Value = 1` (Valor atual; **atenção:** o `Value` inicial deve estar sempre entre `Minimum` e `Maximum`, senão dá erro!).
  - `Dock`, `Anchor`, `Width`, `TabIndex`, `Margin`, `Enabled`.
- **Leitura e Escrita (`Value` é do tipo `decimal`):**
  - **Ler como `int` (obriga cast `(int)`):** `int estoque = (int)numEstoque.Value;`
  - **Ler como `double`:** `double desc = (double)numDesconto.Value;`
  - **Escrever:** `numDependentes.Value = alunoEmEdicao.NumDependentes;`
- **Eventos usados:**
  - `numDesconto.ValueChanged += CalcularPrecoFinal;` (Dispara sempre que o número muda).

Outros controles de escala mencionados na Aula 6 (Conceito Teórico):

- **`TrackBar`**: Barra deslizante para escolher um valor contínuo aproximado (ex: volume).
- **`ProgressBar`**: Barra que comunica visualmente o andamento/progresso de uma operação.

3.5. Ações, Validação e Diálogos (`Button`, `ErrorProvider`, `MessageBox`, `OpenFileDialog`, `PictureBox`)

`Button` (Botão de Ação)

- **Atributos na Criação:** `Text = "Salvar"`, `AutoSize = true`, `TabIndex = 0`, `Anchor = AnchorStyles.Left`, `Enabled = false`.
- **Evento Principal:** `btnSalvar.Click += SalvarAluno;`
- **Dica de Atalho:** `btnFechar.Click += (s, e) => Close();` (Expressão lambda para ações de 1 linha).

`ErrorProvider` (Indicador Visual de Erro)

- **O que é:** Componente **não visual** que exibe um ícone vermelho piscante ao lado de um controle inválido com uma mensagem ao passar o mouse.
- **Passo a passo obrigatório:**
  1. **Declarar campo:** `private readonly ErrorProvider erros = new();`
  2. **Vincular ao Form (no Construtor):** `erros.ContainerControl = this;`
  3. **Limpar erros antes de validar ou no Novo:** `erros.Clear();`
  4. **Acionar erro em um campo:** `erros.SetError(txtNome, "Informe pelo menos 3 caracteres.");`
  5. **Liberar memória no final da classe (`Dispose`):**

```
C#
protected override void Dispose(bool disposing)
{
    if (disposing)
        erros.Dispose();
    base.Dispose(disposing);
}
```

`MessageBox` (Caixa de Diálogo Modal)

- **1. Para exibir um aviso/erro simples (como na Prática 2):**

```
C#
MessageBox.Show("O nome deve ter no mínimo 3 caracteres.", "Erro de Validação",
MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

- **2. Para pedir confirmação antes de Excluir (Padrão de todas as aulas):**

```
C#
DialogResult resposta = MessageBox.Show(
    this,                                   // Janela proprietária
    $"Excluir {alunoEmEdicao.Nome}?",       // Texto da pergunta
    "Confirmar exclusão",                   // Título da caixinha
    MessageBoxButtons.YesNo,                // Botões Sim e Não
    MessageBoxIcon.Question,                // Ícone de interrogação
    MessageBoxDefaultButton.Button2         // Foco padrão no botão 2 ("Não") por
segurança
);

if (resposta != DialogResult.Yes)
    return; // Cancela a exclusão se não clicou em Sim
```

`PictureBox` e `OpenFileDialog` (Exibir e Escolher Imagem)

- **Configuração do `PictureBox`:**

```
C#
private readonly PictureBox picImagem = new()
{
    Dock = DockStyle.Fill,
    SizeMode = PictureBoxSizeMode.Zoom,         // Ajusta a imagem sem distorcer a
proporção!
    BorderStyle = BorderStyle.FixedSingle,      // Borda simples ao redor do quadro
    MinimumSize = new Size(100, 100)
};
```

- **Como carregar ou limpar imagem no `PictureBox`:**
  - Carregar: `picImagem.ImageLocation = caminhoImagemAtual;`
  - Limpar: `picImagem.ImageLocation = null;`
- **Como abrir o seletor de arquivos (`OpenFileDialog`):**

```
C#
private void EscolherImagem(object? sender, EventArgs e)
{
    using OpenFileDialog ofd = new();
    ofd.Filter = "Imagens (*.png;*.jpg;*.jpeg)|*.png;*.jpg;*.jpeg";
    ofd.Title = "Selecione a imagem do produto";
```

```
    if (ofd.ShowDialog() == DialogResult.OK)
    {
        caminhoImagemAtual = ofd.FileName; // Caminho completo do arquivo
escolhido
        picImagem.ImageLocation = caminhoImagemAtual;
    }
}
```

Outros diálogos e componentes não visuais da Aula 6 (Conceito Teórico):

- **`ToolTip`**: Exibe dica ao passar o mouse (`dicas.SetToolTip(txtNome, "Digite o nome completo.");`).
- **`SaveFileDialog`**, **`FolderBrowserDialog`**, **`ColorDialog`**, **`FontDialog`**.
- **`Timer`** (evento `Tick` em intervalos curtos — nunca fazer operações demoradas nele para não travar a tela), **`ImageList`**, **`NotifyIcon`**, **`BackgroundWorker`**.

3.6. Tabelas e Dados (`DataGridView` e `BindingList<T>`)

`BindingList<T>` (Coleção Vinculada em Memória)

- **Namespace:** `using System.ComponentModel;`
- **Declaração:** `private readonly BindingList<Aluno> alunos = new();`
- **Conexão com a grade (no Construtor):** `grade.DataSource = alunos;`
- **Métodos principais:**
  - `alunos.Add(novoAluno);` (Adiciona na lista e atualiza a grade sozinho).
  - `alunos.Remove(alunoEmEdicao);` (Remove da lista e atualiza a grade sozinho).
  - `alunos.ResetItem(alunos.IndexOf(alunoEmEdicao));` (**Crucial na Edição:** força a grade a redesenhar a linha editada).
  - `alunos.Count` (Retorna o total de itens na lista).
- **Evento de mudança na lista:**
  - `alunos.ListChanged += (s, e) => AtualizarStatus();` (Dispara automaticamente sempre que um item é adicionado, editado ou removido!).

`DataGridView` (Grade de Dados)

- **Configuração Completa (Campo da Classe):**

```
C#
private readonly DataGridView grade = new()
{
    Dock = DockStyle.Fill,
    ReadOnly = true,                                        // Impede edição
direta na célula
    TabIndex = 3,
    AllowUserToAddRows = false,                             // Remove a
linha vazia extra no final
    AllowUserToDeleteRows = false,                          // Impede
deletar apertando Delete
    MultiSelect = false,                                    // Permite
selecionar só 1 linha por vez
    RowHeadersVisible = false,                              // Oculta a
coluna cinza lateral esquerda
    SelectionMode = DataGridViewSelectionMode.FullRowSelect,      // Clicar
seleciona a linha inteira
    AutoGenerateColumns = false,                            // true = gera
todas as propriedades; false = você escolhe as colunas
    AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill,   // Estica as
colunas para ocupar toda a largura
    BackgroundColor = SystemColors.Window,
    BorderStyle = BorderStyle.Fixed3D
};
```

- **Adicionando Colunas Manualmente (quando `AutoGenerateColumns = false`):**

```
C#
// DataPropertyName = nome EXATO da propriedade na classe Aluno/Produto
// HeaderText = título que aparece no topo da coluna
grade.Columns.Add(new DataGridViewTextBoxColumn { DataPropertyName = "Nome",
HeaderText = "Nome" });
grade.Columns.Add(new DataGridViewTextBoxColumn { DataPropertyName = "Curso",
HeaderText = "Curso" });
```

- **Métodos Úteis:**
  - `grade.ClearSelection();` (Desmarca qualquer linha selecionada).
- **Eventos da Grade:**
  1. **`CellClick` (Clicar em uma linha para carregar para edição):**

```
C#
private void SelecionarAluno(object? sender, DataGridViewCellEventArgs args)
{
    if (args.RowIndex < 0) return; // Ignora clique no cabeçalho da coluna!

    var aluno = grade.Rows[args.RowIndex].DataBoundItem as Aluno;
    if (aluno is null) return;
```

```
    alunoEmEdicao = aluno;
    // Copia os dados do objeto para os campos da tela...
}
```

2. **`CellFormatting` (Pintar linhas e formatar valores como Moeda — Prática 2):**

```
C#
private void FormatarGradeEstoque(object? sender,
DataGridViewCellFormattingEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        var produto = grade.Rows[e.RowIndex].DataBoundItem as Produto;
        if (produto != null)
        {
            // Muda a cor de fundo da linha inteira conforme condição
            if (produto.Estoque == 0)
                grade.Rows[e.RowIndex].DefaultCellStyle.BackColor =
Color.LightCoral;
            else if (produto.Estoque >= 1 && produto.Estoque <= 5)
                grade.Rows[e.RowIndex].DefaultCellStyle.BackColor =
Color.LightYellow;
            else
                grade.Rows[e.RowIndex].DefaultCellStyle.BackColor = Color.White;

            // Formata coluna específica como Moeda (R$)
            if (grade.Columns[e.ColumnIndex].DataPropertyName == "Preco")
            {
                e.Value = produto.PrecoFinal.ToString("C");
                e.FormattingApplied = true;
            }
        }
    }
}
```

3. **`MouseDown` (Selecionar linha com o Botão Direito do Mouse antes de abrir o `ContextMenuStrip` — Aula 6):**

```
C#
private void SelecionarComBotaoDireito(object? sender, MouseEventArgs e)
{
    if (e.Button == MouseButtons.Right)
    {
        var hit = grade.HitTest(e.X, e.Y);
```

```
        if (hit.RowIndex >= 0)
        {
            grade.ClearSelection();
            grade.Rows[hit.RowIndex].Selected = true;
            SelecionarAluno(grade, new DataGridViewCellEventArgs(hit.ColumnIndex,
hit.RowIndex));
        }
    }
}
```

3.7. Contêineres de Layout (`TableLayoutPanel`, `FlowLayoutPanel` e outros)

`TableLayoutPanel` (Grade de Layout em Linhas e Colunas)

- **Uso:** Organizar toda a janela em células (`coluna, linha`), adaptando ao redimensionamento.
- **Configuração Completa:**

```
C#
var layout = new TableLayoutPanel
{
    Dock = DockStyle.Fill,
    Padding = new Padding(12),
    ColumnCount = 4, // Total de colunas
    RowCount = 6     // Total de linhas
};

// Configurar largura das Colunas:
// SizeType.AutoSize -> Ajusta à largura do maior controle (ótimo para Labels)
// SizeType.Percent, 100 -> Ocupa todo o espaço restante (ótimo para TextBoxes e
Grade)
layout.ColumnStyles.Add(new ColumnStyle(SizeType.AutoSize));
layout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100));

// Configurar altura das Linhas:
// Use AutoSize para linhas de campos/botões e Percent (100) para a linha do
DataGridView!
layout.RowStyles.Add(new RowStyle(SizeType.AutoSize));
layout.RowStyles.Add(new RowStyle(SizeType.Percent, 100));

// Adicionar controles: layout.Controls.Add(controle, COLUNA, LINHA);
layout.Controls.Add(lblNome, 0, 1);
```

```
layout.Controls.Add(txtNome, 1, 1);

// Mesclar colunas (fazer um controle atravessar várias colunas):
layout.SetColumnSpan(txtNome, 3); // Ocupa 3 colunas a partir da coluna onde foi
inserido
```

`FlowLayoutPanel` (Fila Sequencial de Controles)

- **Uso:** Agrupar botões (`Salvar`, `Novo`, `Excluir`) ou agrupar `RadioButtons` lado a lado.
- **Configuração:**

```
C#
var botoes = new FlowLayoutPanel
{
    Dock = DockStyle.Fill,
    AutoSize = true,
    TabIndex = 6,
    FlowDirection = FlowDirection.LeftToRight // Ou FlowDirection.RightToLeft para
alinhar à direita!
};
botoes.Controls.AddRange(new Control[] { btnSalvar, btnNovo, btnExcluir });
```

Outros contêineres da Aula 6:

- **`GroupBox`**: Agrupa controles com uma borda e um título no topo (muito usado para envolver `RadioButtons` ou seções do formulário).
- **`Panel`**: Agrupa controles sem título e permite barra de rolagem com `AutoScroll = true`.
- **`TabControl`** (separa em abas) e **`SplitContainer`** (divide em 2 áreas redimensionáveis).

3.8. Menus e Barras (`MenuStrip`, `ToolStrip`, `ContextMenuStrip`, `StatusStrip`)

1. `MenuStrip` (Barra de Menu Superior: Arquivo, Editar, Ajuda)

```
C#
// Campos da classe:
private readonly MenuStrip menuPrincipal = new();
```

```
private readonly ToolStripMenuItem menuArquivo = new() { Text = "&Arquivo" }; // O
'&' cria atalho Alt+A
private readonly ToolStripMenuItem menuEditar = new() { Text = "&Editar" };
private readonly ToolStripMenuItem menuAjuda = new() { Text = "&Ajuda" };

// Em MontarTela():
menuPrincipal.Items.Add(menuArquivo);
menuPrincipal.Items.Add(menuEditar);
menuPrincipal.Items.Add(menuAjuda);
MainMenuStrip = menuPrincipal;
Controls.Add(menuPrincipal);
```

2. `ToolStrip` (Barra de Ferramentas com Botões Rápidos e Separadores)

```
C#
// Campos da classe:
private readonly ToolStrip barraFerramentas = new();
private readonly ToolStripButton tsbNovo = new() { Text = "Novo", ToolTipText =
"Criar nova inscrição" };
private readonly ToolStripButton tsbSalvar = new() { Text = "Salvar", ToolTipText
= "Salvar inscrição atual" };
private readonly ToolStripButton tsbExcluir = new() { Text = "Excluir",
ToolTipText = "Excluir inscrição selecionada" };

// Em MontarTela():
var separador = new ToolStripSeparator();
var separador2 = new ToolStripSeparator();
barraFerramentas.Items.AddRange(new ToolStripItem[] { tsbNovo, separador,
tsbSalvar, separador2, tsbExcluir });
Controls.Add(barraFerramentas);

// Eventos (em ConectarEventos):
tsbNovo.Click += NovoAluno;
tsbSalvar.Click += SalvarAluno;
tsbExcluir.Click += ExcluirAluno;
```

3. `ContextMenuStrip` (Menu de Clique com Botão Direito na Grade)

```
C#
// Campos da classe:
private readonly ContextMenuStrip menuGrade = new();
private readonly ToolStripMenuItem menuGradeEditar = new() { Text = "Editar" };
private readonly ToolStripMenuItem menuGradeExcluir = new() { Text = "Excluir" };

// Em MontarTela():
```

```
menuGrade.Items.AddRange(new ToolStripItem[] { menuGradeEditar, menuGradeExcluir
});
grade.ContextMenuStrip = menuGrade; // Associa o menu de contexto ao DataGridView!

// Em ConectarEventos():
menuGradeEditar.Click += (s, e) => txtNome.Focus();
menuGradeExcluir.Click += ExcluirAluno;
```

4. `StatusStrip` (Barra de Status no Rodapé da Janela)

```
C#
// Campos da classe:
private readonly StatusStrip barraStatus = new() { Dock = DockStyle.Bottom };
private readonly ToolStripStatusLabel lblStatusTexto = new()
{
    Text = "Pronto",
    Spring = true, // Spring = true faz este label empurrar os próximos para o
canto direito!
    TextAlign = ContentAlignment.MiddleLeft
};
private readonly ToolStripStatusLabel lblContadorInscricoes = new()
{
    Text = "Nenhuma inscrição",
    Spring = false,
    TextAlign = ContentAlignment.MiddleRight
};

// Em MontarTela():
barraStatus.Items.AddRange(new ToolStripItem[] { lblStatusTexto,
lblContadorInscricoes });
Controls.Add(barraStatus);
```

ATENÇÃO — Ordem de Empilhamento (`BringToFront`) em `MontarTela()`:
Quando você usa `MenuStrip`, `ToolStrip`, `StatusStrip` e um `TableLayoutPanel` com `Dock = DockStyle.Fill`, adicione-os ao `Form` e use `BringToFront()` no `layout` por último para que o painel principal não fique escondido atrás das barras:

```
C#
Controls.Add(barraFerramentas);
Controls.Add(menuPrincipal);
menuPrincipal.BringToFront();
Controls.Add(barraStatus);
Controls.Add(layout);
```

```
layout.BringToFront(); // Garante que o TableLayoutPanel respeite o espaço das barras!
```

---

4. Receitas Prontas de Lógica (Para Copiar/Adaptar na Prova)

4.1. Ciclo de Vida de Estados (Inclusão vs. Edição)

- **`alunoEmEdicao == null`** → Modo **Inclusão** (Botão diz `"Salvar"`, cria um `new Aluno()` e usa `alunos.Add(...)`).
- **`alunoEmEdicao != null`** → Modo **Edição** (Botão diz `"Atualizar"`, altera as propriedades de `alunoEmEdicao` e chama `alunos.ResetItem(alunos.IndexOf(alunoEmEdicao))`).

4.2. Validações Clássicas de Prova (`ValidarCampos`)

```
C#
private bool ValidarCampos()
{
    erros.Clear();

    // 1. Validar tamanho mínimo de texto
    if (txtNome.Text.Trim().Length < 3)
    {
        erros.SetError(txtNome, "Informe pelo menos 3 caracteres.");
        txtNome.Focus();
        return false;
    }

    // 2. Validar se escolheu item no ComboBox
    if (cmbCurso.SelectedIndex < 0)
    {
        erros.SetError(cmbCurso, "Selecione um curso.");
        cmbCurso.Focus();
        return false;
    }

    // 3. Validar código/matrícula repetida (desconsiderando o próprio item em edição!)
    string codigoDigitado = txtCodigo.Text.Trim();
    foreach (Produto p in produtos)
    {
        if (p.Codigo == codigoDigitado && p != produtoEmEdicao)
        {
```

```
            erros.SetError(txtCodigo, "Já existe um registro com este código.");
            txtCodigo.Focus();
            return false;
        }
    }

    // 4. Validar número decimal (Preço) positivo
    if (!double.TryParse(txtPreco.Text.Trim(), out double precoValor) ||
precoValor < 0)
    {
        erros.SetError(txtPreco, "Preço inválido.");
        txtPreco.Focus();
        return false;
    }

    return true;
}
```

4.3. Filtro / Pesquisa em Tempo Real na Grade

```
C#
private void PesquisarProduto(object? sender = null, EventArgs? args = null)
{
    string termo = txtPesquisa.Text.Trim().ToLower();
    string categoria = cmbCategoria.SelectedItem?.ToString() ?? "";
    bool somenteEstoque = chkAtivo.Checked;

    BindingList<Produto> filtrados = new();

    foreach (Produto p in produtos)
    {
        bool atendeTexto = string.IsNullOrEmpty(termo) ||
                           p.Nome.ToLower().Contains(termo) ||
                           p.Codigo.ToLower().Contains(termo);
        bool atendeCat = categoria == "Todas" || p.Categoria == categoria;
        bool atendeEst = !somenteEstoque || p.Estoque > 0;

        if (atendeTexto && atendeCat && atendeEst)
            filtrados.Add(p);
    }

    grade.DataSource = filtrados;
}
```